TP9 - DevSecOps y observabilidad de PhotoMatch

1. Objetivo y teoría

Shift-left adelanta detección: Dependabot vigila dependencias, CodeQL analiza flujos de código y push protection intercepta secretos conocidos. Monitoreo externo mide disponibilidad; Sentry muestra errores, tiempos y uso. Un dato observado debe generar una acción en el tablero.

2. Situación inicial

No había Dependabot ni CodeQL, aunque secret scanning y push protection ya estaban activos. La app no tenía Sentry ni monitor externo. El fallback local todavía no había llegado a PROD desde el pipeline.

3. Archivos modificados

.github/dependabot.yml, **backend/src/instrument.js**, **backend/src/app.js**, **backend/src/routes/index.js**, **backend/package*.json**, **frontend/src/telemetry.js**, **frontend/package*.json**, **frontend/Dockerfile**, **frontend/default.conf.template** y la cabecera EJS.

4. Código importante

Dependabot cubre npm backend, npm frontend y GitHub Actions. CodeQL usa default setup de GitHub. El backend inicializa **@sentry/node** antes de Express si **SENTRY_DSN** existe y registra el manejador de errores. El navegador carga un bundle de **@sentry/browser**; **/telemetry-config.js** le entrega DSN de ingestión y SHA desde variables del entorno. Si no hay DSN, funciona sin Sentry. El bundle se construye con esbuild durante el build del front.

5. Decisiones y alternativas

No se versionan DSN de backend ni credenciales de lectura. El DSN del navegador es público por diseño: permite enviar eventos, no leerlos. No se creó una alerta falsa ni una tarjeta sin datos. El monitor externo aún requiere una URL alcanzable fuera de la máquina; **localhost** no sirve a UptimeRobot. La cuenta y el túnel o hosting siguen pendientes.

6. Herramientas

Dependabot crea PRs de versión y seguridad; CodeQL busca patrones de vulnerabilidad sin ejecutar tests; push protection compara patrones de secrets; Sentry recibe eventos y spans; un monitor externo consulta **/health** y alerta DOWN/UP.

7. Tests y evidencia real

GitHub confirma Dependabot security updates y push protection activos. El primer CodeQL default setup pasó en run 40272141414; el escaneo del PR detectó dos alertas altas que se corrigieron; la nueva corrida y su check terminaron verdes. **npm audit fix** resolvió cuatro avisos transitivos; audit posterior: cero. Los builds y 33 unitarios, 3 integración y 3 E2E pasan. **Faltan** PR de Dependabot mergeado, Sentry con error y métricas reales, DOWN/UP externo y pausa, tarjeta de Project y release **v9.0.0**.

8. Problemas y solución

El backend arrastraba cuatro dependencias vulnerables, incluida una crítica en **proxy-addr**. **npm audit fix** actualizó el lockfile y se repitieron los tests. CodeQL encontró un regex de email con backtracking y un **/health** sin límite: la validación ahora es lineal y **express-rate-limit** permite hasta 60 peticiones por minuto. El frontend de PhotoMatch es Nginx más EJS, sin bundle JS previo; se agregó esbuild sólo para instrumentación de navegador.

9. Preguntas del profesor

¿Qué no ve CodeQL? Comportamiento real del servicio y secretos fuera del código. **¿Qué no ve el monitor?** Formularios rotos si **/health** sigue verde. **¿Qué agrega Sentry?** Error con contexto y distribución de tiempos/uso. **¿Por qué pausar UptimeRobot?** En un hosting gratuito evita consumir cuota despertando servicios; con fallback local se debe coordinar la demo.

10. Demo de defensa

Abrir Security para CodeQL, Dependabot y push protection, mostrar PR Dependabot verde, monitor externo con DOWN/UP y pausa, Sentry con error y operaciones más lenta/usada, y tarjeta del Project nacida de una cifra real. Estas evidencias todavía no existen y deben completarse antes de declarar TP9 terminado.